



Full Name: Aishwarya S
Email: aishwaryasriramu@gmail.com
Test Name: Mock Test
Taken On: 28 Aug 2025 11:09:32 IST
Time Taken: 12 min 51 sec/ 90 min
Invited by: Ankush
Invited on: 28 Aug 2025 11:09:21 IST
Skills Score:
Tags Score:

100%
290/290

scored in **Mock Test** in 12 min
51 sec on 28 Aug 2025 11:09:32
IST

- Algorithms 290/290
- Arrays 95/95
- Core CS 290/290
- Data Structures 215/215
- Easy 95/95
- Medium 75/75
- Queues 120/120
- Search 75/75
- Sorting 95/95
- Strings 95/95
- problem-solving 170/170

Recruiter/Team Comments:

No Comments.

Plagiarism flagged

We have marked questions with suspected plagiarism below. Please review it in detail here -

	Question Description	Time Taken	Score	Status
Q1	Truck Tour > Coding	26 sec	120/ 120	!
Q2	Pairs > Coding	33 sec	75/ 75	!
Q3	Big Sorting > Coding	32 sec	95/ 95	!

QUESTION 1

Truck Tour > Coding

Algorithms

Data Structures

Queues

Core CS



Needs Review

Score 120

QUESTION DESCRIPTION

Suppose there is a circle. There are N petrol pumps on that circle. Petrol pumps are numbered 0 to $(N - 1)$ (both inclusive). You have two pieces of information corresponding to each of the petrol pump: (1) the amount of petrol that particular petrol pump will give, and (2) the distance from that petrol pump to the next petrol pump.

Initially, you have a tank of infinite capacity carrying no petrol. You can start the tour at any of the petrol pumps. Calculate the first point from where the truck will be able to complete the circle. Consider that the truck will stop at each of the petrol pumps. The truck will move one kilometer for each litre of the petrol.

Input Format

The first line will contain the value of N .

The next N lines will contain a pair of integers each, i.e. the amount of petrol that petrol pump will give and the distance between that petrol pump and the next petrol pump.

Constraints:

$$1 \leq N \leq 10^5$$

$$1 \leq \text{amount of petrol, distance} \leq 10^9$$

Output Format

An integer which will be the smallest index of the petrol pump from which we can start the tour.

Sample Input

```
3
1 5
10 3
3 4
```

Sample Output

```
1
```

Explanation

We can start the tour from the second petrol pump.

CANDIDATE ANSWER

Language used: C

```
1
2  /*
3   * Complete the 'truckTour' function below.
4   *
5   * The function is expected to return an INTEGER.
6   * The function accepts 2D_INTEGER_ARRAY petrolpumps as parameter.
7   */
8
9  int truckTour(int petrolpumps_rows, int petrolpumps_columns, int**
10 petrolpumps) {
11  long long total_petrol = 0;
12  long long curr_petrol = 0;
13  int start_index = 0;
14
15  for(int i=0; i<petrolpumps_rows; i++){
16      int petrol = petrolpumps[i][0];
17      int distance = petrolpumps[i][1];
```

```

18     total_petrol += (petrol - distance);
19     curr_petrol += (petrol - distance);
20
21     if(curr_petrol < 0){
22         start_index = i+1;
23         curr_petrol = 0;
24     }
25 }
26
27 if(total_petrol >= 0){
28     return start_index;
29 }else{
30     return -1;
31 }
32 }

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Sample case	✔ Success	0	0.0072 sec	7.25 KB
Testcase 2	Easy	Hidden case	✔ Success	10	0.0077 sec	7 KB
Testcase 3	Easy	Hidden case	✔ Success	10	0.008 sec	6.75 KB
Testcase 4	Easy	Hidden case	✔ Success	10	0.007 sec	7.25 KB
Testcase 5	Easy	Hidden case	✔ Success	10	0.0333 sec	17 KB
Testcase 6	Easy	Hidden case	✔ Success	10	0.0396 sec	16.9 KB
Testcase 7	Easy	Hidden case	✔ Success	10	0.04 sec	17 KB
Testcase 8	Easy	Hidden case	✔ Success	10	0.0409 sec	17 KB
Testcase 9	Easy	Hidden case	✔ Success	10	0.0398 sec	16.8 KB
Testcase 10	Easy	Hidden case	✔ Success	10	0.032 sec	17 KB
Testcase 11	Easy	Hidden case	✔ Success	10	0.0406 sec	16.6 KB
Testcase 12	Easy	Hidden case	✔ Success	10	0.0403 sec	16.9 KB
Testcase 13	Easy	Hidden case	✔ Success	10	0.0363 sec	17 KB

No Comments

QUESTION 2



Needs Review

Score 75

Pairs > Coding

Search

Algorithms

Medium

problem-solving

Core CS

QUESTION DESCRIPTION

Given an array of integers and a target value, determine the number of pairs of array elements that have a difference equal to the target value.

Example

$k = 1$

$arr = [1, 2, 3, 4]$

There are three values that differ by $k = 1$: $2 - 1 = 1$, $3 - 2 = 1$, and $4 - 3 = 1$. Return **3**.

Function Description

Complete the *pairs* function below.

pairs has the following parameter(s):

- int k*: an integer, the target difference
- int arr[n]*: an array of integers

Returns

- *int*: the number of pairs that satisfy the criterion

Input Format

The first line contains two space-separated integers *n* and *k*, the size of *arr* and the target value.

The second line contains *n* space-separated integers of the array *arr*.

Constraints

- $2 \leq n \leq 10^5$
- $0 < k < 10^9$
- $0 < arr[i] < 2^{31} - 1$
- each integer *arr[i]* will be unique

Sample Input

STDIN	Function
5 2	arr[] size n = 5, k =2
1 5 3 4 2	arr = [1, 5, 3, 4, 2]

Sample Output

3

Explanation

There are 3 pairs of integers in the set with a difference of 2: [5,3], [4,2] and [3,1]. .

CANDIDATE ANSWER

Language used: C

```
1
2 /*
3  * Complete the 'pairs' function below.
4  *
5  * The function is expected to return an INTEGER.
6  * The function accepts following parameters:
7  * 1. INTEGER k
8  * 2. INTEGER_ARRAY arr
9  */
10
11 int compare(const void* a, const void* b){
12     int int_a = *(int*)a;
13     int int_b = *(int*)b;
14     return (int_a - int_b);
15 }
16
17 int pairs(int k, int arr_count, int* arr) {
18     qsort(arr, arr_count, sizeof(int), compare);
19
20     int count = 0;
21     int i = 0, j=1;
22
23     while(j<arr_count){
24         int diff = arr[j] - arr[i];
25
26         if(diff == k){
27             count++;
28             i++;
29             j++;
30         }
31     }
```

```

30     }else if(diff < k){
31         j++;
32     }else{
33         i++;
34         if(i == j){
35             j++;
36         }
37     }
38 }
39 return count;
40 }
41

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Hidden case	✔ Success	5	0.0073 sec	7.25 KB
Testcase 2	Easy	Hidden case	✔ Success	5	0.0086 sec	7.38 KB
Testcase 3	Easy	Hidden case	✔ Success	5	0.0072 sec	7.38 KB
Testcase 4	Easy	Hidden case	✔ Success	5	0.0108 sec	6.88 KB
Testcase 5	Easy	Hidden case	✔ Success	5	0.0098 sec	6.88 KB
Testcase 6	Easy	Hidden case	✔ Success	5	0.0084 sec	7.38 KB
Testcase 7	Easy	Hidden case	✔ Success	5	0.0086 sec	7.38 KB
Testcase 8	Easy	Hidden case	✔ Success	5	0.0071 sec	7.13 KB
Testcase 9	Easy	Hidden case	✔ Success	5	0.0092 sec	7.25 KB
Testcase 10	Easy	Hidden case	✔ Success	5	0.0093 sec	7.25 KB
Testcase 11	Easy	Hidden case	✔ Success	5	0.0403 sec	9.38 KB
Testcase 12	Easy	Hidden case	✔ Success	5	0.0341 sec	9.38 KB
Testcase 13	Easy	Hidden case	✔ Success	5	0.0334 sec	9.38 KB
Testcase 14	Easy	Hidden case	✔ Success	5	0.0353 sec	9.29 KB
Testcase 15	Easy	Hidden case	✔ Success	5	0.0339 sec	9.39 KB
Testcase 16	Easy	Sample case	✔ Success	0	0.0082 sec	7 KB
Testcase 17	Easy	Sample case	✔ Success	0	0.0078 sec	7.25 KB
Testcase 18	Easy	Sample case	✔ Success	0	0.0101 sec	7.38 KB

No Comments

QUESTION 3



Needs Review

Score 95

Big Sorting > Coding

Sorting

Strings

Algorithms

Easy

Data Structures

Arrays

problem-solving

Core CS

QUESTION DESCRIPTION

Consider an array of numeric strings where each string is a positive number with anywhere from **1** to **10^6** digits. Sort the array's elements in *non-decreasing*, or ascending order of their integer values and return the sorted array.

Example

unsorted = ['1', '200', '150', '3']

Return the array ['1', '3', '150', '200'].

Function Description

Complete the *bigSorting* function in the editor below.

bigSorting has the following parameter(s):

- *string unsorted[n]*: an unsorted array of integers as strings

Returns

- *string[n]*: the array sorted in numerical order

Input Format

The first line contains an integer, *n*, the number of strings in *unsorted*.

Each of the *n* subsequent lines contains an integer string, *unsorted[i]*.

Constraints

- $1 \leq n \leq 2 \times 10^5$
- Each string is guaranteed to represent a positive integer.
- There will be no leading zeros.
- The total number of digits across all strings in *unsorted* is between **1** and **10^6** (inclusive).

Sample Input 0

```
6
31415926535897932384626433832795
1
3
10
3
5
```

Sample Output 0

```
1
3
3
5
10
31415926535897932384626433832795
```

Explanation 0

The initial array of strings is

unsorted = [31415926535897932384626433832795, 1, 3, 10, 3, 5]. When we order each string by the real-world integer value it represents, we get:

$$1 \leq 3 \leq 3 \leq 5 \leq 10 \leq 31415926535897932384626433832795$$

We then print each value on a new line, from smallest to largest.

Sample Input 1

```
8
1
2
100
12303479849857341718340192371
3084193741082937
3084193741082938
111
200
```

Sample Output 1

```
1
2
100
111
200
3084193741082937
3084193741082938
12303479849857341718340192371
```

CANDIDATE ANSWER

Language used: C

```
1
2 /*
3  * Complete the 'bigSorting' function below.
4  *
5  * The function is expected to return a STRING_ARRAY.
6  * The function accepts STRING_ARRAY unsorted as parameter.
7  */
8
9 /*
10 * To return the string array from the function, you should:
11 *     - Store the size of the array to be returned in the result_count
12 variable
13 *     - Allocate the array statically or dynamically
14 *
15 * For example,
16 * char** return_string_array_using_static_allocation(int* result_count) {
17 *     *result_count = 5;
18 *
19 *     static char* a[5] = {"static", "allocation", "of", "string", "array"};
20 *
21 *     return a;
22 * }
23 *
24 * char** return_string_array_using_dynamic_allocation(int* result_count) {
25 *     *result_count = 5;
26 *
27 *     char** a = malloc(5 * sizeof(char*));
28 *
29 *     for (int i = 0; i < 5; i++) {
30 *         *(a + i) = malloc(20 * sizeof(char));
31 *     }
32 *
33 *     *(a + 0) = "dynamic";
34 *     *(a + 1) = "allocation";
35 *     *(a + 2) = "of";
36 *     *(a + 3) = "string";
37 *     *(a + 4) = "array";
38 *
39 *     return a;
40 * }
41 *
42 */
43
44 int compare(const void* a, const void* b){
45     char* str1 = *(char**)a;
46     char* str2 = *(char**)b;
47
48     int len1 = strlen(str1);
```

```

49     int len2 = strlen(str2);
50
51     if(len1 != len2){
52         return len1-len2;
53     }
54
55     return strcmp(str1, str2);
56 }
57
58 char** bigSorting(int unsorted_count, char** unsorted, int* result_count) {
59
60     qsort(unsorted, unsorted_count, sizeof(char*), compare);
61
62     *result_count = unsorted_count;
63     return unsorted;
64 }

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Sample case	✔ Success	0	0.0064 sec	7 KB
Testcase 2	Medium	Hidden case	✔ Success	10	0.0079 sec	7 KB
Testcase 3	Medium	Hidden case	✔ Success	10	0.0124 sec	7.88 KB
Testcase 4	Hard	Hidden case	✔ Success	15	0.016 sec	8.38 KB
Testcase 5	Hard	Hidden case	✔ Success	15	0.0242 sec	8.38 KB
Testcase 6	Hard	Hidden case	✔ Success	15	0.0229 sec	8.38 KB
Testcase 7	Hard	Hidden case	✔ Success	15	0.0357 sec	9.15 KB
Testcase 8	Hard	Hidden case	✔ Success	15	0.1303 sec	15.8 KB
Testcase 9	Easy	Sample case	✔ Success	0	0.0081 sec	7.25 KB

No Comments